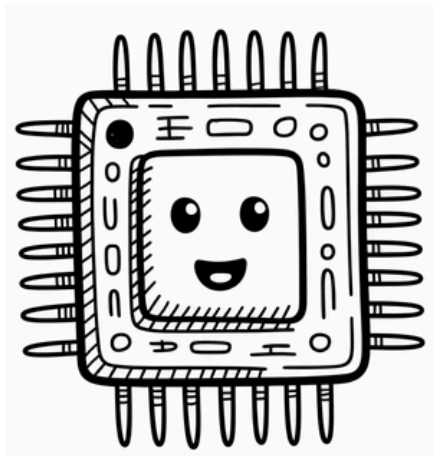
AI Accelerator Program

Analytics Vidhya

A Guide to Context Engineering

Andrej Karpathy

puts it



LLMs are like a **new kind of operating system**

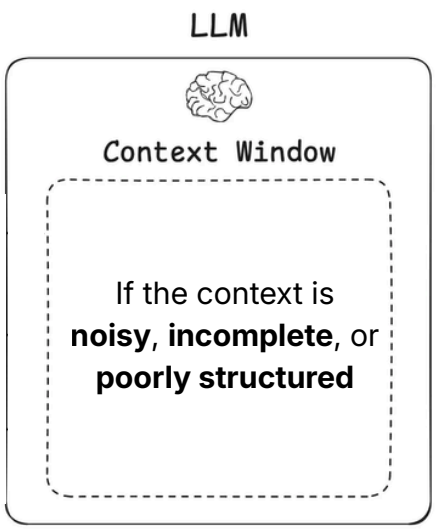
LLM is like the

CPU

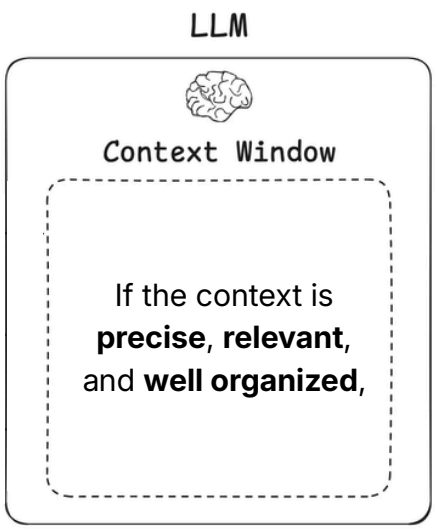
its **Context Window** is like

RAM

serving as the model's working memory.



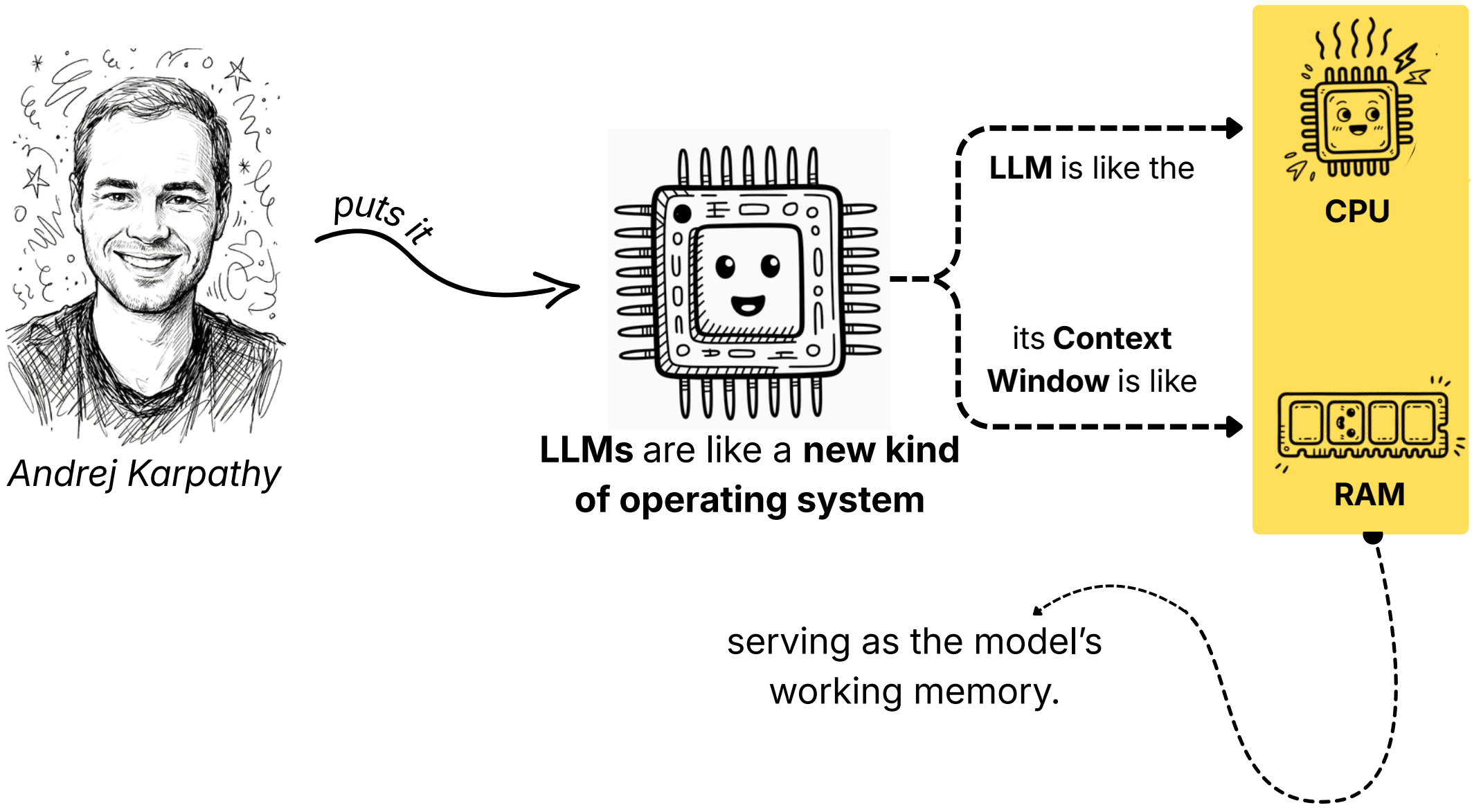
The **output quality degrades**



The **model behaves intelligently**

Context engineering is essentially memory management for LLM systems. It sits at the core of:

- **RAG**
- **Autonomous workflows**
- **Chatbots**
- **AI agents**
- **Tool-using systems**



Context engineering is the delicate art and science of filling the context window with just the right information for the next step.

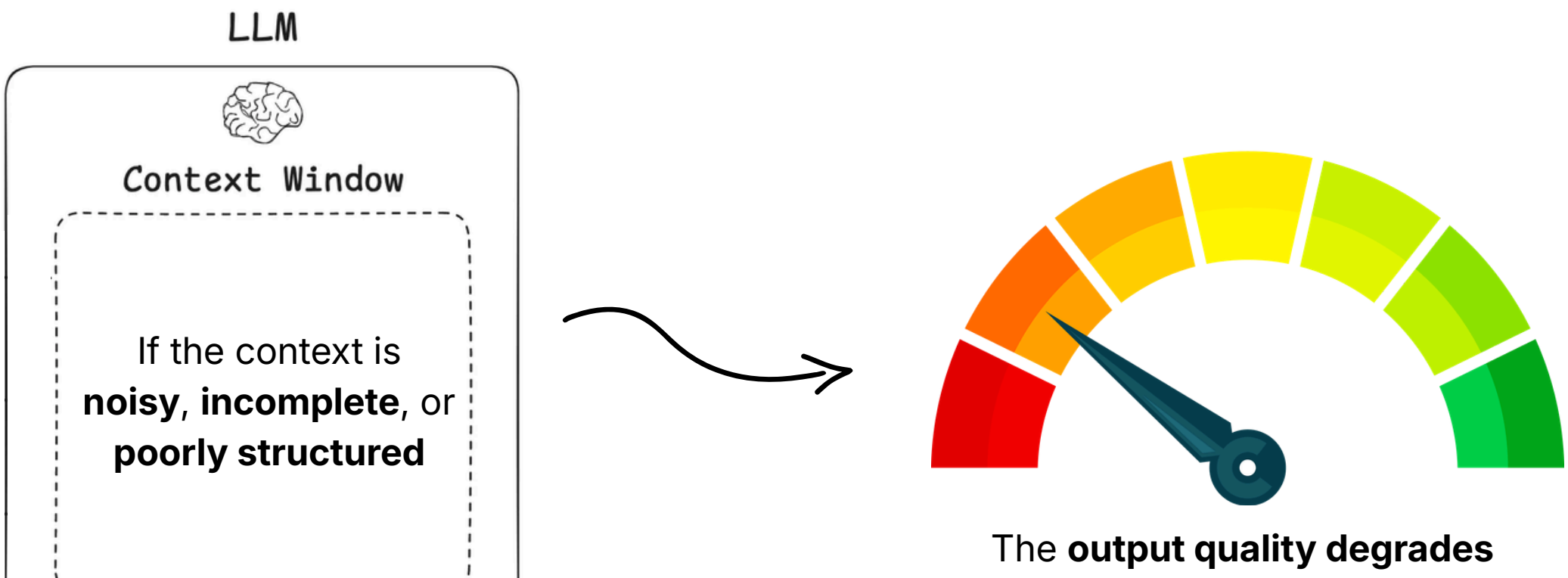
A large language model does not have persistent memory. It only knows what exists inside its current context window, which includes:

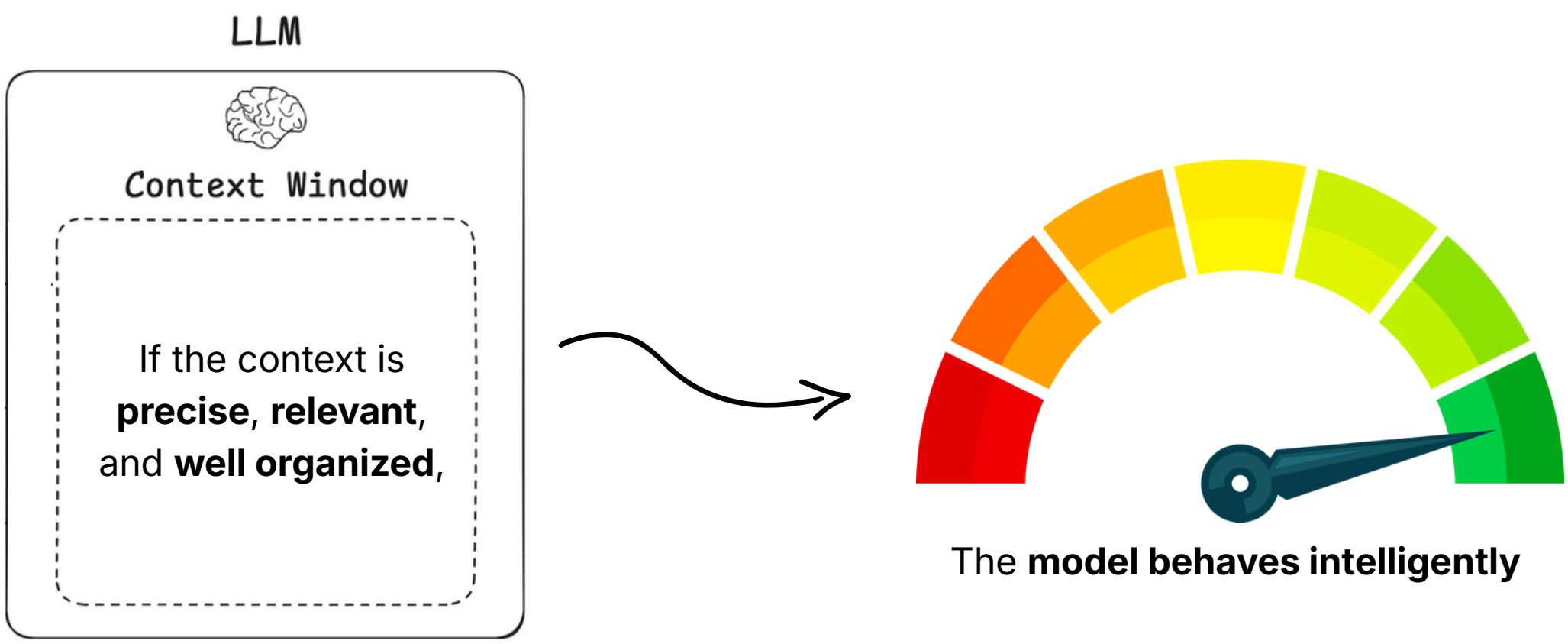
System prompt	User message	Conversation history
Tool outputs	Retrieved documents	Structured state

Everything the model says is a function of this context.

Formally, you can think of it like this:

$$Output = LLM(Context)$$

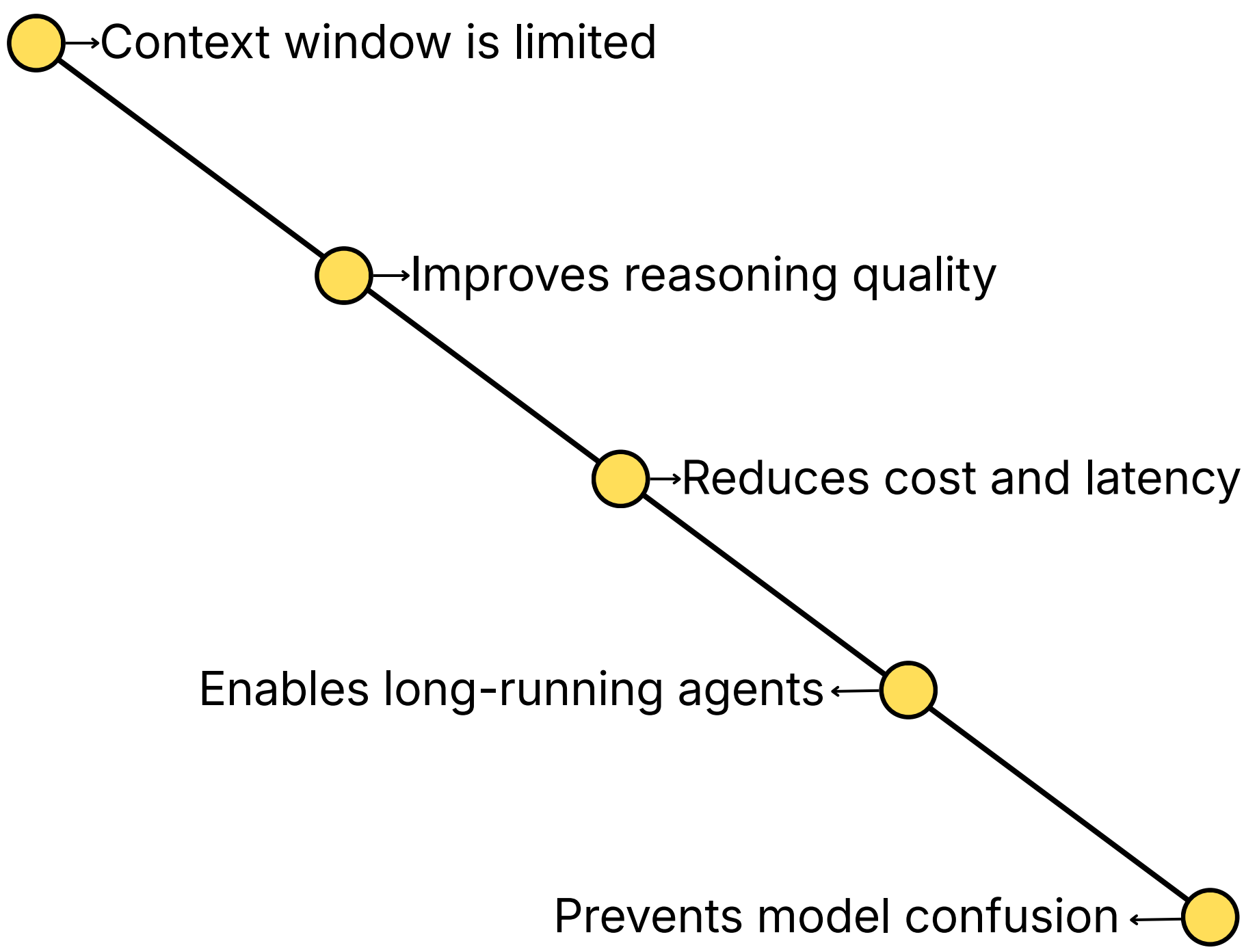




Context engineering is essentially memory management for LLM systems. It sits at the core of:

- **AI agents**
- **Chatbots**
- **RAG**
- **Tool-using systems**
- **Autonomous workflows**

Why Context Engineering is important?



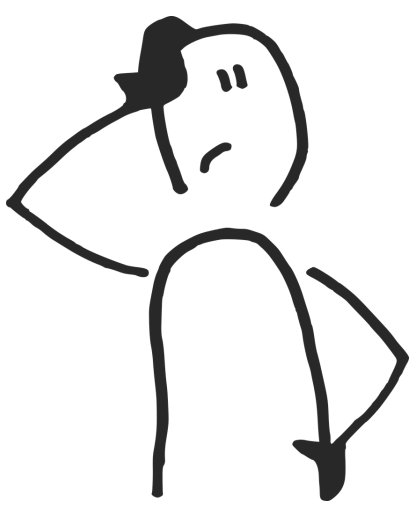
Context window is limited

Models have finite context windows:

- 8K tokens
- 32K tokens
- 128K tokens
- 200K tokens (some frontier models)

Without engineering, context fills quickly and breaks.

Problems that appear:



Lost instructions

Forgetting earlier goals

Repeating questions

Hallucinations

Context engineering prevents context overflow

Improves reasoning quality

→ LLMs reason based on visible information.
→ Irrelevant context introduces noise.
→ Signal-to-noise ratio matters.

Bad Context

- 50 irrelevant messages
- 2 relevant ones

Good Context

- Only relevant messages
- Structured summaries

Result → **Better Reasoning**

Reduces cost and latency

Token cost is proportional to context size.

Less context = ⬆⬆⬆ Faster responses
⬇⬇⬇ Lower API cost
⬆⬆⬆ Better scalability

Enables long-running agents

Agents that run for hours or days require memory compression.
Context engineering enables:

Persistent memory

State tracking

Goal continuity

Prevents model confusion

Unfiltered tool outputs and logs overwhelm the model.

Context engineering filters and structures state.

Core Methods for Context Engineering

Summarization of History Messages

Problem

Conversation grows continuously.

Example:
User asks 100 questions.
Passing all 100 messages every time becomes inefficient.
And older messages become less relevant.

Solution

Replace older messages with a compressed summary.

Example:

Before:
User: Build ML pipeline
Assistant: Sure
User: Use XGBoost
Assistant: Ok
User: Add cross validation
Assistant: Done
User: Add feature selection
Assistant: Done

After summarization:
Summary:
User is building ML pipeline using:

- XGBoost
- Cross validation
- Feature selection

Now instead of 10 messages, you pass 1 summary message.

Filtering Out Older History Messages

Not all messages deserve summarization.

Many are useless.

Example:

- Hello
- Thanks
- Ok
- Yes

These messages add zero value.

Strategy: Relevance filtering

Remove messages that do not affect reasoning.
Relevance scoring methods:

Method 1: Heuristic filtering

Remove:

- Short acknowledgements
- Greetings
- Confirmations

Method 2: Recency filtering

Keep only last N messages.
Example:
Keep last 10 messages.
Discard rest.

Method 2: Embedding similarity filtering

Step 1:Convert messages into embeddings.
Step 2:Compare similarity with current query.
Step 3:Keep only relevant ones.

Summarization of Tool Calls

This is extremely important for AI agents. Tool outputs can be huge. Example:

Search tool returns:10,000 tokens
Logs return: 20,000 tokens
Database results return: Thousands of rows

Passing raw outputs destroys context.

Solution: Summarize tool outputs

Convert tool outputs into structured summaries.

Example

Raw output:
Database returned 500 customer records with fields name, age, purchase, date.

Solution: Summarize tool outputs

Summary:

Database contains:

- 500 customers
- Age range: 18-65
- Avg purchase: $250

Now model can reason efficiently.

Real-world Example

AI Coding Assistants	(Cursor, GitHub Copilot) Tools like Cursor and GitHub Copilot analyze entire codebases, mapping out dependencies, project structure, and code relationships to tailor responses to your work
Insurance Claims at Five Sigma	Five Sigma Insurance achieved an 80% reduction in claim processing errors and a 25% increase in adjustor productivity by architecting AI systems that ingest policy data, claims history, and regulations simultaneously
Healthcare Virtual Assistants	Healthcare virtual assistants with context engineering now consider patients' health records, medication schedules, and live appointment tracking delivering accurate, safe advice and dramatically reducing administrative overhead